

Java 面试 Q&A 整理

一、Java 核心基础

Q1: HashMap 扩容原理

1.7 vs 1.8 区别:

版本	数据结构	扩容时机	扩容方式	扩容大小
1.7	数组+链表	$\text{size} > \text{threshold} \ \&\& \text{当前桶非空}$	头插法 (多线程死循环)	2倍
1.8	数组+链表+红黑树	$\text{size} > \text{threshold}$	尾插法	2倍

1.8 扩容过程:

1. 当 $\text{size} > \text{loadFactor} * \text{capacity}$ (默认 0.75) 时触发扩容
2. 容量变为原来的 2 倍
3. 重新计算每个元素的位置——不需要重新 hash, 只需判断原 hash 值新增 bit 是 0 还是 1:
 - 0 → 原索引不变
 - 1 → 原索引 + oldCap
4. 链表长度 ≥ 8 且数组长度 ≥ 64 时, 链表转红黑树
 1. 某个桶达到 8 时:
 1. 数组长度小于 64, 只**扩容**, 不**树化**
 2. 大于 64, 转为红黑树
5. 扩容后重新分配时, 红黑树节点 ≤ 6 时退化为链表

扩容过程:

假设容量从 4 扩容到 8, 某个 key 的 hash 值二进制为 ...0101:

- 旧索引 = $0101 \ \& \ 0011 = 0001$ (即 1)
- 判断 $\text{hash} \ \& \ \text{oldCap} = 0101 \ \& \ 0100 = 0100$ (不等于 0)
- 新索引 = 原索引 + oldCap = 1 + 4 = 5

如果另一个 key 的 hash 为 ...0001:

- $\text{hash} \ \& \ \text{oldCap} = 0 \rightarrow \text{新索引} = \text{原索引} = 1。$

追问: 1000 条数据往初始容量 16 的 HashMap 放, 会扩容几次?

- 扩容阈值 = $16 \times 0.75 = 12$
- 扩容路径: 16 → 32 (12 满) → 64 (24 满) → 128 (48 满) → 256 (96 满) → 512 (192 满) → 1024 (384 满)
- 1000 条数据落在 1024 容量内, **共扩容 6 次**

延伸:

- 为什么负载因子是 0.75? 空间和时间的折中, 太低浪费内存, 太高碰撞概率大
- 为什么容量是 2 的幂? 方便位运算 $(n-1) \ \& \ \text{hash}$ 代替取模

- ConcurrentHashMap 扩容：多线程协同迁移，每个线程负责一段槽位
-

Q2: String 底层实现

- JDK 8 及之前：char[] 数组，每个 char 占 2 字节
- JDK 9+：byte[] + coder 标记，Latin-1 字符用 1 字节，节省内存（Compact Strings）
- jdk9：coder区分Latin-1和utf-16

```
// "hello" 和 "he" + new String("llo") 比较
String a = "hello";           // 字符串常量池
String b = "he" + new String("llo"); // 运行时拼接, new String在堆中
// a == b → false (a在常量池, b在堆中)
// a.equals(b) → true
```

延伸——字符串拼接原理：

- 编译期常量拼接 → 编译器优化为常量（"a"+"b" 直接编译为 "ab"）
 - 变量拼接 → 使用 StringBuilder.append() 或 StringConcatFactory
 - 循环中拼接：务必显式使用 StringBuilder，否则每次循环创建新实例
-

Q3: int vs Integer 比较（拆箱与装箱）

```
int a = 100;
Integer b = 100;
// a == b → true (Integer自动拆箱为int, 值比较)

Integer c = 200;
Integer d = 200;
// c == d → false (超出缓存池 -128~127, 各自new对象)
// 默认缓存上限可通过 -XX:AutoBoxCacheMax=size 调整
```

IntegerCache 机制： Integer 内部类 IntegerCache，默认缓存 [-128, 127] 的 Integer 对象。valueOf() 方法在范围内返回缓存对象，超出范围 new 新对象。Byte/Short/Long 也有类似缓存，范围固定 [-128, 127]。为什么 Integer 要做 IntegerCache？：目的是节省内存、提升性能。范围内这些数使用频率高。避免使用 new Integer 创建，使用 integer i = 10; 这种调用 valueOf() 自动装箱。

Q4: 多线程 run() vs start()

```
new Thread(() -> System.out.println("A")).run(); // 当前线程执行，同步
new Thread(() -> System.out.println("B")).start(); // 新线程执行，异步
```

- run()：当前线程直接调用方法体，不会创建新线程
- start()：JVM 创建新线程，新线程调用 run()。一个线程只能 start 一次，再次调用抛 IllegalStateException

延伸——线程生命周期（6 种状态）： NEW → RUNNABLE → BLOCKED / WAITING / TIMED_WAITING → TERMINATED

Q5: 单例模式手写

```
// DCL（双重检查锁）——最经典的线程安全懒汉式
public class Singleton {
    // volatile 防止指令重排（分配内存 → 初始化 → 赋值引用，可能乱序）
    private static volatile Singleton instance;

    private Singleton() {}

    public static Singleton getInstance() {
        if (instance == null) { // 第一重检查（避免加锁开销）
            synchronized (Singleton.class) {
                if (instance == null) { // 第二重检查（防止重复创建）
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}
```

延伸：

- 饿汉式：类加载时初始化，线程安全但可能浪费内存
- 枚举式：天然防反射、防序列化攻击（Effective Java 推荐）
- Spring 单例 Bean 默认是单例，但 Spring 管理的"单例"是容器级别的（一个 BeanDefinition 一个实例）

Q6: 集合数据去重方法

场景	方法
List 去重	<code>new ArrayList<>(new HashSet<>(list))</code> （无序）； <code>ls.stream().distinct().toList()</code>
List 去重保持顺序	<code>new ArrayList<>(new LinkedHashSet<>(list))</code>
按对象某个字段去重	<code>list.stream().filter(distinctByKey(User::getId)).collect(toList())</code>
自定义去重（TreeSet）	<code>new TreeSet<>(Comparator.comparing(User::getId))</code>

```
// 按字段去重的工具方法
private static <T> Predicate<T> distinctByKey(Function<? super T, ?> keyExtractor) {
    Set<Object> seen = ConcurrentHashMap.newKeySet();
    return t -> seen.add(keyExtractor.apply(t));
}
```

Q7: List 转 Map

```
// 基本转换
Map<Long, User> map = list.stream()
    .collect(Collectors.toMap(User::getId, Function.identity()));
```

```
// key 冲突处理：保留旧值
Map<Long, User> map2 = list.stream()
    .collect(Collectors.toMap(User::getId, u -> u, (old, nu) -> old));

// 按字段分组
Map<Long, List<User>> group = list.stream()
    .collect(Collectors.groupingBy(User::getDeptId));
```

注意：Collectors.toMap() key 重复默认抛 IllegalStateException，必须指定 mergeFunction。

Q8: 多线程如何保证线程安全

线程安全问题本质是多个线程同时访问共享资源导致的数据不一致。

可以根据实际场景，按下面的思路来选择方案：

1. 能不共享吗？ -> 优先使用局部变量。
2. 能直接使用现成的安全类吗？ -> 使用 ConcurrentHashMap, Atomic* 等。
3. 必须自己控制锁吗？ -> 优先使用 synchronized，代码更简单；需要超时、中断等高级功能时，再考虑 ReentrantLock。
4. 是想“隔离”而非“共享”吗？ -> 使用 ThreadLocal。

任何同步机制都会带来或多或少的性能开销，基本原则是：只在必要时保护数据，尽量缩小同步代码块的范围。

方式	说明
synchronized	JVM 内置锁，自动加/解锁，悲观锁
ReentrantLock（可重入锁）	API 层面，支持公平锁、条件变量、可中断
volatile	保证可见性+禁止指令重排，不保证原子性
Atomic 类	CAS 无锁，适合计数/状态位
ThreadLocal	线程私有副本，空间换安全;使用完必须手动remove()
不可变对象	final 修饰，天然线程安全
并发集合	ConcurrentHashMap、CopyOnWriteArrayList

线程生命周期：

将线程的生命周期与生活中的场景关联起来，会更容易记忆：

- **NEW**：你刚拿到一张招聘启事。
- **RUNNABLE**：你投了简历，正在等待HR筛选（就绪）或正在面试（运行）。
- **BLOCKED**：会议室（锁）被占用，你被挡在门外等别人出来。
- **WAITING**：面试结束了，你回家等通知（无限期等待），直到HR给你打电话（notify）。
- **TIMED_WAITING**：HR让你等10分钟，她去找主管确认（超时等待）。
- **TERMINATED**：你收到offer并入职了；或者被明确告知不合适，流程结束。

Q9: 线程池核心线程数如何设计

公式（《Java 并发编程实战》）：

任务类型	公式
CPU 密集型(加密，排序等)	$N_CPU + 1$
IO 密集型（网络调用、数据库读写、文件 IO）	$N_CPU * 2$ 或 $N_CPU / (1 - \text{阻塞系数})$ ，阻塞系数 $\approx 0.8 \sim 0.9$

任务类型

混合型

公式

拆分为 CPU 密集 + IO 密集两个线程池

实际调整因子：

- 考虑其他服务共用机器 → 适当降低
- 考虑任务平均耗时、峰值 QPS → $\text{coreSize} = \text{QPS} \times \text{avgRT} / 1000$
- 核心线程数 = 期望 QPS × 单任务耗时（秒）

```
// 示例：期望 QPS=200，单个任务 50ms
// coreSize = 200 × 0.05 = 10
```

延伸——线程池参数拒绝策略：

- AbortPolicy（默认）：抛异常
- CallerRunsPolicy：由调用线程执行
- DiscardPolicy：静默丢弃
- DiscardOldestPolicy：丢弃队列中最旧任务

****动态线程池（Dynamic Thread Pool）：****它的核心思想极其粗暴且优雅：

1. **不要写死参数：**把 `corePoolSize` 和 `maximumPoolSize` 丢到配置中心（比如 Nacos、Apollo）。
2. **利用 Java 原生 API 动态修改：**`ThreadPoolExecutor` 其实自带了 `setCorePoolSize(int)` 和 `setMaximumPoolSize(int)` 方法！
3. **监控 + 联动：**写个定时任务或者结合监控系统，一旦发现线程池的队列堆积率超过 80%，或者活跃线程数长期触顶，直接在配置中心把核心线程数调大，**不需要重启服务器，线上秒级生效！**
4.

```
// 不能在运行时直接修改 corePoolSize，但可以 set
executor.setCorePoolSize(newCoreSize);
executor.setMaximumPoolSize(newMaxSize);
```

Q10: 死锁怎么造成？如何避免？

四个必要条件（缺一不可）：

1. 互斥：资源独占
2. 持有并等待：持有锁 A 等锁 B
3. 不可剥夺：不能强行抢锁
4. 循环等待：A→B→C→A 的等待环

避免方法：

- **破坏循环等待：**统一加锁顺序，按固定顺序获取锁；比如，根据账户的 ID 大小来加锁，永远先锁 ID 小的，再锁 ID 大的！
- **破坏持有并等待：**一次性申请所有锁（如 `tryLock` 超时）；
- **破坏不可剥夺：**使用 `ReentrantLock.tryLock(timeout, unit)`
- **检测工具：**jstack 查看线程状态，查找 BLOCKED 和 waiting to lock
- **数据库死锁：**缩短事务、固定访问顺序、合理索引减少锁范围
- 能用局部变量的就别用全局变量（还记得咱们聊过的 `ThreadLocal` 吗？）。
- 能用无锁原子类（如 `AtomicInteger`）的就别用 `synchronized`。
- 如果非要加锁，尽量只在需要改数据的那一两行代码上加，快进快出，绝不拖泥带水。

Q11: LocalDateTime.now() 线程安全吗?

线程安全。与 SimpleDateFormat (非线程安全) 不同:

- LocalDateTime、LocalDate、Instant 等 Java 8 时间类都是不可变对象, 天然线程安全
- DateTimeFormatter 也是不可变+线程安全的
- SimpleDateFormat → 必须用 ThreadLocal 包装或加锁
- Date 可变的 setTime() → 不安全

延伸: 导出文件名用

LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyyMMddHHmmss")) 是安全的。

二、Spring 框架

Q12: Spring Bean 是否线程安全?

默认不保证线程安全。

- Spring 不提供线程安全策略, 线程安全性由 Bean 自身的设计决定
- 无状态 Bean (如 Service/Controller 中不存成员变量): 天然线程安全
- 有状态 Bean (持有可变成员变量): 非线程安全, 需自行控制
- prototype 作用域: 每次创建新实例, 不等于线程安全, 只是减少了共享
- @Scope("request"): 每个请求一个实例, 实际上是 ThreadLocal 实现

最佳实践: Controller/Service 层保持无状态, 必要状态数据存入 ThreadLocal 或 Redis。

Q13: @Transactional 失效的三大原因

1. 方法不是 public

- Spring AOP 基于动态代理 (JDK/CGLIB), 只能代理 public 方法
- 私有/受保护方法的 @Transactional 直接无效

2. 同类自调用

```
@Service
public class UserService {
    public void methodA() {    // 外部调用, 有事务
        this.methodB();        // this 是原始对象, 不是代理, 事务不生效!
    }

    @Transactional
    public void methodB() { }
}
```

解决方案: 注入自己 (@Autowired private UserService self), 通过 self.methodB() 调用。

3. 异常被捕获

```
@Transactional
public void doSomething() {
```

```

try {
    dbOperation();          // 抛异常
} catch (Exception e) {
    log.error("error", e); // 事务不回滚！
}
}

```

- `@Transactional` 默认回滚策略: `RuntimeException` 和 `Error` 回滚, 受检异常不回滚
- 指定回滚: `@Transactional(rollbackFor = Exception.class)`
- 异常被 `catch` 吞掉后 Spring 感知不到, 自然不会回滚

延伸: `@Transactional` 的 `propagation` 传播行为 — `REQUIRED` (默认)、`REQUIRES_NEW`、`NESTED` 等。

Q14: Spring 事务执行机制

1. **代理创建:** Spring 为 `@Transactional` 的 Bean 生成 AOP 代理对象 (proxy)
2. **获取连接:** 代理拦截方法调用 → 从 `DataSource` 获取数据库连接 → 关闭自动提交 (`setAutoCommit(false)`)
3. **绑定连接:** 当前连接绑定到 `ThreadLocal` (`TransactionSynchronizationManager`)
4. **执行业务:** 同一个事务内所有 SQL 共用同一条连接
5. **提交/回滚:** 正常完成 → `commit`; 异常判断 `rollbackFor` → `rollback`
6. **释放连接:** 解绑 `ThreadLocal`, 归还连接池

关键: 同一个事务内多次数据库操作使用的必须是同一个 `Connection` (`ThreadLocal` 保证)。

spring事务三个核心:

TransactionInterceptor (事务拦截器): 这是个大特务 (AOP切面)。它每天就在系统里巡逻, 盯着谁的方法上贴了 `@Transactional`。一旦发现有人调用它, 立刻冲上去把方法“拦截”住。

PlatformTransactionManager (事务管理器): 这是真正的纯干活的大佬 (比如专门管 JDBC/MyBatis 的 `DataSourceTransactionManager`)。它负责去跟数据库连线、开事务、提事务、滚事务。

TransactionSynchronizationManager (事务同步管理器): 这是个神奇的百宝箱。因为它底层全是 **ThreadLocal** (还记得咱们聊过各玩各的吧?)。它负责把数据库连接 (`Connection`) 死死绑定在当前线程上, 保证你在同一个事务里执行的七八个 Mapper 用的都是**同一个数据库连接**。

Q15: Spring MVC 和 Spring Boot 区别

维度	Spring MVC	Spring Boot
定位	Web 框架 (MVC 实现)	快速开发脚手架
配置	大量 XML/Java 配置	自动配置 + Starter
部署	需要外部 Servlet 容器 (Tomcat)	内嵌 Tomcat, java -jar 运行
依赖管理	手动管理版本	版本仲裁 (spring-boot-dependencies)
关注点	只做 MVC	基础设施全家桶 (监控/健康检查/外部化配置)
关系	Spring Boot 内置了 Spring MVC	Spring Boot = Spring MVC + 自动配置 + Starter + Actuator

延伸——内嵌 Tomcat 原理: Spring Boot 启动时创建 `TomcatServletWebServerFactory` → `new Tomcat` 实例 → 注册 `DispatcherServlet` → 启动监听端口。jar 包之所以是 web 应用, 是因为内嵌了 Servlet 容器。

Q16: @SpringBootApplication 注解

```
@SpringBootApplication // 三合一注解
├─ @SpringBootConfiguration // = @Configuration, Java 配置类
├─ @EnableAutoConfiguration // 自动配置, 通过 spring.factories 加载
xxxAutoConfiguration
├─ @ComponentScan           // 扫描当前包及子包下的组件
```

自动配置原理:

1. @EnableAutoConfiguration → @Import(AutoConfigurationImportSelector.class)
2. 读取 META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
3. 按条件注解 (@ConditionalOnClass、@ConditionalOnBean、@ConditionalOnMissingBean 等) 过滤
4. 符合条件则创建 Bean 注入容器

Q17: Spring Security 认证授权

核心流程:

1. 认证 (Authentication) : 用户 → Filter Chain → AuthenticationManager → ProviderManager → UserDetailsService 查库 → 比对密码 (BCryptPasswordEncoder) → 返回 Authentication 对象存入 SecurityContextHolder
2. 授权 (Authorization) : 请求 → FilterSecurityInterceptor → 比对所需角色/权限 → 通过则放行

核心组件:

- SecurityContextHolder: 通过 ThreadLocal 持有当前用户信息
- BCryptPasswordEncoder: 加盐哈希, 每次加密生成随机盐, 结果不同
- JWT + Spring Security: 自定义 OncePerRequestFilter 解析 Token → 构建 Authentication → 存入 SecurityContextHolder

```
// BCryptPasswordEncoder 特点: 同一密码每次加密结果不同 (随机盐)
String hash1 = encoder.encode("123456"); // $2a$10$xxx1
String hash2 = encoder.encode("123456"); // $2a$10$xxx2 (不同!)
// matches 方法解析盐值后比对
encoder.matches("123456", hash1); // true
```

三、Spring Cloud 微服务

Q18: OpenFeign 跨服务调用全链路

注解使用:

```
// 主类
@EnableFeignClients
@SpringBootApplication
public class Application {}

// 接口定义
@FeignClient(name = "order-service", path = "/api/orders",
```

```

        fallbackFactory = OrderClientFallbackFactory.class) // 配置了fallback, 这个
服务宕机就走fallback的实现
    public interface OrderClient {
        @GetMapping("/{id}")
        Result<Order> getById(@PathVariable Long id);

        @PostMapping
        Result<Void> create(@RequestBody OrderDTO dto);
    }

```

底层调用逻辑：

1. @EnableFeignClients 扫描 @FeignClient 接口
2. JDK 动态代理生成接口实现类
3. 调用方法时 → MethodHandler 构建 RequestTemplate (URL/Method/Headers/Body)
4. LoadBalancer 从 Nacos 获取服务列表，负载均衡选择实例
5. 实际 HTTP：默认 HttpURLConnection，推荐替换为 Apache HttpClient (连接池)
6. 响应解码器 → 转换回方法返回类型
7. 熔断：Sentinel/Hystrix 保护，降级走 fallback

Feign 调用失败排查：

- 检查 Nacos 服务是否在线
- 检查 @FeignClient name 是否与服务名一致
- 检查超时配置 (feign.client.config.xxx.connectTimeout/readTimeout)
- 开启 Feign 日志 logging.level.com.xxx.client.OrderClient=DEBUG
- 使用 Arthas trace 追踪完整调用链

feign 每次发 http 请求建立连接问题： 配置 Apache HttpClient 连接池，复用 TCP 连接

Q19: 微服务体系核心组件

组件	功能	选型
注册中心	服务注册/发现	Nacos / Eureka
配置中心	动态配置	Nacos Config
网关	统一入口	Spring Cloud Gateway
远程调用	服务间通信	OpenFeign
熔断降级	防止级联故障	Sentinel
负载均衡	请求分发	Spring Cloud LoadBalancer

四、MySQL 数据库

Q20: MySQL 底层 B+ 树

为什么用 B+ 树而不是 B 树/红黑树/哈希？

对比	B+ 树	B 树	红黑树	哈希
数据存储	仅在叶子节点	非叶节点也存	节点存单值	数组+链表
叶子节点	双向链表连接	无连接	-	-
范围查询	极快 (遍历链表)	需中序遍历	需遍历	不支持
高度	矮 (一个节点存16KB)	矮	高	-

对比	B+ 树	B 树	红黑树	哈希
IO 次数	少	少	多	一次

B+ 树结构：

- 非叶子节点只存 key + 子节点指针（16KB 一页可存约 1200 个索引键）
- 叶子节点存完整行数据（聚簇索引）或主键值（二级索引）
- 叶子节点之间用双向指针链接，支持正序/倒序扫描

Q21: 回表是什么？

回表：通过二级索引查询时，先在二级索引 B+ 树找到主键值，再回到聚簇索引 B+ 树查完整行数据的过程。

查询：SELECT * FROM user WHERE name = '张三'; -- name 有索引

Step 1: 走 name 索引 → 叶子节点找到 name='张三' 对应的主键 id=100

Step 2: 拿 id=100 回聚簇索引 → 找到完整行数据 ← 这就是回表

优化——覆盖索引：如果查询的列都在二级索引中，直接返回，不回表。

```
CREATE INDEX idx_name_age ON user(name, age);
SELECT name, age FROM user WHERE name = '张三'; -- 覆盖索引，不回表
```

Q22: 联合索引与最左匹配原则

最左匹配：联合索引 (a, b, c) 按 a → b → c 的顺序建立 B+ 树，查询条件必须从最左列开始才能走索引。

联合索引：idx_abc(a, b, c)

WHERE a=1	→ 走索引 (a)
WHERE a=1 AND b=2	→ 走索引 (a+b)
WHERE a=1 AND b=2 AND c=3	→ 走索引 (完整)
WHERE a=1 AND c=3	→ a 走索引, c 不走 (跳过 b)
WHERE b=2	→ 不走索引 (缺 a)
WHERE b=2 AND c=3	→ 不走索引 (缺 a)
WHERE a=1 AND b>2 AND c=3	→ a+b 走索引, c 不走 (范围后的列失效)
WHERE a>20 AND b=1 AND c=2	→ 只有 a 走索引 (a 是范围查询, b/c 失效)

追问：【联合索引 idx(a, b, c)，SQL = WHERE a>20 AND b=1 ORDER BY c DESC 索引怎么走？】

- **过滤阶段：**a 范围查询 → 只用到 a (b=1 是等值但 a 已是范围，所以 b 不能用于过滤)
- **排序阶段：**因为 a 是范围查询，跳过了 b，索引 (a, c) 不是有序的，排序也用不上索引
- **结论：**仅 a 走索引过滤，ORDER BY c 走 filesort

IN 导致索引失效的情况：

- MySQL 5.5 之前 IN 完全不走索引
- MySQL 5.6+ IN 走索引，但条件是优化器根据数据分布（基数）决定的。大量 IN 值 → 优化器可能认为全表扫描更快 → 强制索引用 FORCE INDEX

Q23: INT(10) 和 INT(11) 的区别

核心结论：没有任何区别。

- 括号里的数字是**显示宽度**，不是存储大小
- INT 固定 4 字节存储，范围 -2147483648 ~ 2147483647
- 显示宽度只在 **ZEROFILL** 时生效：INT(3) ZEROFILL 插入 1 → 显示 001
- MySQL 8.0.17 开始 display width 已**废弃**

INT(1)	INT(5)	INT(10)	INT(11)
--------	--------	---------	---------

最大值相同：2147483647	相同	相同	相同
------------------	----	----	----

Q24: ORDER BY 什么情况索引失效？

1. 排序字段不在索引中 → filesort
2. 排序字段顺序与索引不匹配（如索引 (a,b)，ORDER BY b）→ filesort
3. 升序降序混用与索引排序不一致 → filesort
4. 排序带有非索引列的函数/表达式 → filesort
5. 走了一个索引，ORDER BY 用了另一个索引列 → filesort（MySQL 只会选一个索引）
6. WHERE 是范围查询，ORDER BY 不在同一索引中 → filesort

Q25: SQL 数据量大、有索引还是很慢怎么办？

1. 确认索引真正命中：EXPLAIN 看 type（ALL/INDEX 说明没走或全索引扫）
2. 选错索引：EXPLAIN 看 possible_keys 和 key 不一致 → FORCE INDEX
3. 回表太大：建覆盖索引消除回表
4. limit 深翻页：用"游标法"替代 offset → WHERE id > last_id LIMIT n
5. 数据量确实过大（千万级）：分库分表（ShardingSphere）、引入 ES、冷热分离
6. 慢查询日志：slow_query_log + pt-query-digest 分析
7. 锁竞争：SHOW ENGINE INNODB STATUS 看是否有锁等待

数据量大，有索引还很慢，按这个顺序排雷：

1. 查执行计划 (EXPLAIN)：看type, key, Extra。扫一眼有没有索引失效、没用上覆盖索引？（八股文基本功）
2. 查实际扫描行数 (EXPLAIN ANALYZE)：优化器预估的行数和实际差别大吗？（MySQL 8.0+ 神器）
3. 查锁与事务：是不是被别的会话阻塞了？（信息_schema）
4. 查硬件和配置：Buffer Pool 是不是太小？磁盘是不是瓶颈？
5. 查SQL本身：是不是深度分页？是不是 SELECT * 一把梭？
6. 灵魂拷问：这个需求，适合用MySQL吗？

带头大哥不能死，中间兄弟不能断 列上有函数，索引变废物 类型不对，全表遭罪 Like一开头，索引必定丢 范围一出现，右列全不见 非此即彼，索引难起 OR两边，都得是索引，否则全表扫描安排

看SQL先EXPLAIN，type字段定乾坤。最少要到range级，看到ALL就发晕。key_len量用量，最左前缀断没断。extra里找祸根，filesort和temp最烦人。rows虽然不准确，数量级上见真章。

Q26: 千万级数据查询优化

策略	说明
----	----

分库分表	水平拆分，ShardingSphere / MyCat
------	-----------------------------

读写分离	主库写入，从库查询
------	-----------

策略	说明
ES 同步	Canal 监听 binlog 同步到 ES，搜索走 ES
冷热分离	近期数据在 MySQL，历史数据归档到 Hive/TiDB
分区表	MySQL 原生 partition，按时间范围分区
只查必要列	SELECT col1,col2 而非 SELECT *
缓存预热	Redis 缓存热点数据

Q27: 分库分表

分库分表方式：

- **垂直分库**：按业务拆分（用户库、订单库、商品库）
- **垂直分表**：按字段拆分（主表存常用字段，扩展表存大字段）
- **水平分库**：按分片键 hash 分到不同库
- **水平分表**：同一库内按规则拆分为多个表

分片键选择原则：选择最频繁的查询条件（如 userId、orderId），避免跨库 join/事务。

带来的问题：

- 分布式 ID 生成（雪花算法）
- 跨库 join → 应用层聚合或冗余存储
- 跨库事务 → 柔性事务（Seata/可靠消息）
- 扩容 → 一致性哈希、双倍扩容

五、Redis 缓存

Q28: Redisson 分布式锁 + 看门狗机制

基本用法：

```
RLock lock = redissonClient.getLock("lock:report:" + reportId);
lock.lock(); // 不传过期时间 → 看门狗启动
try {
    generateReport(); // 耗时不定的业务
} finally {
    lock.unlock(); // finally 释放，同时停止看门狗
}
```

看门狗原理：

- 默认过期时间 30s，每 10s (=30s/3) 续期一次
- 续期通过 Netty 定时任务执行 Lua 脚本 pexpire
- 续期前检查 EXPIRATION_RENEWAL_MAP 中锁是否还存在（unlock 时会 remove）
- 触发条件：lock() 不传 leaseTime 或传 -1
- 传了具体 leaseTime → **不启动看门狗**，到期即删

网络问题导致看门狗失效：

- 场景：GC 停顿或网络分区 → 续期指令发不到 Redis → key 30s 过期 → 其他线程拿到锁
- 影响：出现两个线程同时持有“锁”，数据不一致

- 缓解：
 - 适当增大 `lockWatchdogTimeout`（配置类设置）
 - 业务层加乐观锁/版本号做二次校验
 - 使用 Redisson 的红锁（RedLock）多节点保证（生产环境较少使用）

Redisson vs Jedis vs Lettuce： | 特性 | Jedis | Lettuce | Redisson | |-----|-----|-----|-----| | 连接方式 | 同步 | 异步（Netty） | 异步（Netty） | | API 风格 | Redis 原生命令 | 原生命令 | 高级 Java 对象（Lock/Map/Queue） | | 分布式锁 | 需自实现 | 需自实现 | 内置看门狗 + 可重入 |

Q29: Redisson 实现报告生成功能（场景题）

场景： 前端点击"生成报告"，后端异步生成，防止重复提交。

用户 → 前端口"生成报告"
 → 后端 GET /report/generate/{id}
 → Redis SETNX "lock:report:{id}"（1分钟内同一报告不能重复生成）
 → 成功：提交 XXL-Job 异步任务
 → 返回 "报告生成中，请稍后刷新"
 → 前端轮询 GET /report/status/{id}（每 3s 一次）
 → 生成完成 → 返回下载链接

```
public Result generateReport(Long reportId) {
    RLock lock = redissonClient.getLock("lock:report:" + reportId);
    // tryLock: 尝试 0 秒获取，拿不到立刻返回，不等待
    if (!lock.tryLock()) {
        return Result.fail("报告正在生成中，请勿重复提交");
    }
    try {
        // 提交异步任务
        xxlJobHelper.submitTask("reportGenerateJob", reportId);
        return Result.ok("已提交生成");
    } finally {
        // 注意：不要立刻 unlock！这里还没生成完。
        // 应该让任务完成后回调 unlock
        // 或者用 SETNX 做提交去重，不用锁控制执行过程
    }
}
```

更好的方案： 用 Redis SETNX 做请求去重，不阻塞；任务状态用 Redis key `report:status:{id}` 标记（PENDING/PROCESSING/DONE/FAILED）。

Q30: 缓存不一致问题——失效模式 vs 双写模式

Cache Aside（旁路缓存）——推荐：

读：先读缓存 → 命中返回 | 未命中 → 查 DB → 写缓存 → 返回
 写：先更新 DB → 再删除缓存

为什么是删缓存而不是更新缓存？

- 更新缓存可能浪费（更新后一直没人读）
- 并发下更新顺序问题可能导致脏数据
- 删除 + 下次读时重建是更安全的模式

延时双删：先删缓存 → 更新 DB → 等待（如 200ms）→ 再删一次缓存

- 解决：更新 DB 期间，其他线程读到旧数据写入了缓存

双写模式（不推荐）：同时更新 DB 和缓存 → 操作非原子，数据可能不一致

最终一致性方案：

- Canal 监听 binlog → 异步更新/删除缓存
 - MQ 保证消息投递 → 消费端更新缓存
-

六、消息队列 (MQ)

Q31: MQ 消息堆积怎么办？

原因排查：

1. 消费者处理能力不足（最常见）
2. 消费者挂了，没有消费
3. 生产端流量突增
4. 消费者代码性能问题（慢查询等）

解决方案（优先级从高到低）：

1. **临时扩容：**增加消费者实例数量
 2. **提高消费并发度：**增大 concurrency / 线程池
 3. **批量消费：**一次拉取多条消息，攒批处理
 4. **跳过非关键消息：**降级策略，丢弃部分日志/通知类消息
 5. **消费端性能优化：**SQL 优化、批量写库、异步化、缓存
 6. **迁移到新 Topic：**紧急扩容新队列，旧消息慢慢迁移
 7. **限制生产者：**上游增加限流
-

Q32: MQ 重复消费 & 消费成功保证

重复消费原因：

- 生产端：网络超时重试
- 消费端：消费完没来得及 ACK 就挂了，重启后重新投递

消费成功保证（ACK 机制）：

- 自动 ACK: `spring.rabbitmq.listener.simple.acknowledge-mode=auto`，方法正常结束自动确认
- 手动 ACK: `channel.basicAck(deliveryTag, false)`，更可控

幂等消费方案： | 方案 | 实现 | |-----|-----| 数据库唯一键 + 插入 | 用 msgId 做唯一键，重复消息插入失败 catch 忽略 || Redis SETNX | SETNX msg:consumed:{msgId} 1，消费前去重 || 版本号乐观锁 | UPDATE xxx SET status=2 WHERE id=1 AND status=1，判断影响行数 || 流水表 | 消费前查消费记录表，已存在则跳过 |

Q33: 消息大量 unack 是什么原因?

unack = 消费者拿到消息但还没确认。

- 通常原因: 消费速度 < 生产速度, 消费者来不及处理
- 影响: unack 消息持续积压 + 新消息进来 → MQ 内存/磁盘告警
- 解决: 扩充消费者、优化消费逻辑、设置合理的 prefetch count 限制每次获取量
- `spring.rabbitmq.listener.simple.prefetch=250` (默认) → 如果处理慢, 调小可减少单消费者积压

七、MyBatis 相关

Q34: MyBatis 一对多怎么查

主键必须使用 `<id>`, MyBatis 通过 `<id>` 判断是否是同一对象, 否则会把每条关联表记录都当作新对象。

```
<!-- 方式一: collection 嵌套结果映射 (推荐, 一次查询) -->
<resultMap id="DeptWithUsers" type="Dept">
  <id property="id" column="dept_id"/>
  <result property="name" column="dept_name"/>
  <collection property="users" ofType="User">
    <id property="id" column="user_id"/>
    <result property="name" column="user_name"/>
  </collection>
</resultMap>

<select id="getDeptWithUsers" resultMap="DeptWithUsers">
  SELECT d.id AS dept_id, d.name AS dept_name,
         u.id AS user_id, u.name AS user_name
  FROM dept d LEFT JOIN user u ON d.id = u.dept_id
  WHERE d.id = #{id}
</select>
```

两种方式:

- 嵌套结果 (JOIN): 一次查询, resultMap 处理映射
- 嵌套查询 (子查询): N+1 问题, 不推荐

八、性能优化与高并发

Q35: 接口很慢怎么优化?

排查链路:

1. 定位慢在哪一层
 - ├ 前端 → Network 面板看请求耗时
 - ├ 网关 → 日志看转发耗时
 - ├ 负载均衡 → 是否路由到异常节点
 - ├ 应用层 → Arthas trace 追踪方法耗时
 - ├ 数据库 → Druid 慢 SQL 监控 / EXPLAIN
 - └ 第三方 → 调用超时设置 / 异步化

2. 针对性优化

- SQL 慢 → 加索引 / 优化语句 / 加缓存
- 循环调 DB → 批量查询 / JOIN
- 串行改并行 → CompletableFuture
- 数据量大 → 分页 / 游标
- 远程调用多 → 并行调用 / 减少链路
- 计算密集 → 异步化 / 预计算

顺着走一遍：

1. **定界（前端/网络/后端）**：先看Network面板，确定是后端响应慢（TTFB长）还是传输慢。
2. **定位（后端哪部分慢）**：看SkyWalking链路，找到耗时最长的那个服务。
3. **定责（代码/数据库/中间件）**：在这个服务上用Arthas的 `t trace`，找到耗时最长的方法。
4. **止血**：
 - **DB问题**：EXPLAIN -> 加索引/覆盖索引/干掉 `SELECT *` -> 分页优化。
 - **锁问题**：杀长事务，改逻辑顺序。
 - **代码问题**：批量化、异步化、并行化。
 - **架构问题**：上缓存、读写分离、消息队列。
5. **验证**：上线后用SkyWalking对比前后的响应时间，用数据说话，别用“我感觉”

Q36: 接口要求 QPS 200 怎么做

逐步拆解：

- QPS 200 = 每秒 200 个请求，每个请求处理窗口 5ms
- **系统承载力**：如果单接口 50ms，最少需要 10 个并发线程 ($200 \times 0.05 = 10$)
- **压测验证**：JMeter/Gatling 压测找瓶颈
- **缓存**：热点数据 Redis 缓存
- **异步**：非核心逻辑消息队列异步处理
- **限流**：Sentinel/Redis 滑动窗口防止超量打垮系统

Q37: POI 导入数据量过大导致 OOM

原因：XSSFWorkbook 全量加载到内存 + 大量 String 对象（每个单元格都是 String 对象）

解决：

- **SAX 模式（XSSF + SAX）**：基于事件驱动流的式读取，逐行解析不占内存
- **EasyExcel**：阿里开源，底层 SAX 模式，使用 listener 逐行处理
- **分批提交**：每 1000 行 insert 一次，用完 gc
- **异步化**：上传文件到 OSS → 后台任务处理 → 通知结果

POI有两种模式：

模式	代表类	内存占用	适用场景
UserModel（用户模型）	XSSFWorkbook	全部加载到内存	小文件（<10MB）
EventModel（事件模型）	XSSFWorkbook + SAX	几乎不占内存	大文件（>10MB）

Q38: 100w 用户积分每年 1.1 清零（场景题）

分析：100w 用户 × 一次 update 全部清零 = 锁表/主从延迟/耗时极长。

方案：

1. **不物理清零**：加年度字段 `year_2025_points`，查询时只查当前年；缺点每年增加一个字段，或者使用一个json。
2. **如果必须清零**：分批更新，每批 5000 条，`WHERE id BETWEEN ? AND ?`，任务间隔 100ms，持续数小时完成
3. **XXL-Job 定时**：1月1日凌晨 2:00 启动分片任务
4. **Redis 缓存年度积分**：清零前预存 Redis，防止瞬时流量打到 DB
5. 提前几天清理不活跃的用户积分

九、场景设计 & 杂项

Q39: 接口调用失败的重试机制

```
// Spring Retry
@Retryable(maxAttempts = 3, backoff = @Backoff(delay = 1000, multiplier = 2))
public Result callExternalApi() { }

// 自定义：指数退避 + 熔断器
int retry = 0;
while (retry < 3) {
    try { return doCall(); }
    catch (Exception e) {
        retry++;
        if (retry == 3) throw e;
        Thread.sleep(1000 * (long) Math.pow(2, retry)); // 2s, 4s
    }
}
```

关键点：必须是幂等操作才能重试；非幂等（如扣款）需要查询确认后重试。

Q40: 接口安全验证

- 认证：JWT Token / OAuth2 / Session
- 防篡改：请求参数签名（MD5/SHA）+ 时间戳防重放
- 防伪造：HTTPS + 密钥 + 非对称加密
- 限流：IP + 用户多维度限制
- 敏感数据：脱敏（身份证/手机号中间打*）
- SQL 注入：参数化查询（MyBatis 用 `#{}` 不用 `${}` ）
- XSS：输出编码/过滤

Q41: Arthas trace 排查接口慢

```
# 安装
curl -O https://arthas.aliyun.com/arthas-boot.jar
java -jar arthas-boot.jar

# 追踪方法调用链及每个节点耗时
trace com.xxx.controller.ReportController generateReport -n 5

# 监控方法出入参和返回值
watch com.xxx.service.ReportService generate "{params,returnObj,throwExp}"
```

```
# 查看方法调用路径
stack com.xxx.service.ReportService generate
```

Q42: Linux 前端调不到后端 Controller

排查链路（从前到后）：

1. 前端 F12 → Network → 确认请求发出，看状态码
 - └─ 404: URL 错误或没注册
 - └─ 500: 后端报错，看日志
 - └─ CORS 错误：跨域没配
2. 后端日志 → `tail -f app.log | grep "请求路径"`
3. `netstat -tlnp | grep 8080` → 确认端口监听
4. `curl localhost:8080/api/xxx` → 本机测试
5. `telnet <ip> 8080` → 检查网络连通性
6. `iptables -L -n` → 检查防火墙规则
7. nginx 层 → `cat /var/log/nginx/error.log`

Q43: WPS 在线编辑接口

一般指 WPS 开放平台的 WebOffice SDK：

- 前端加载 WPS WebOffice 组件，传入 fileId
- 后端提供三个回调接口：获取文件信息、保存文件、获取用户信息
- 文件存储在 OSS/COS 上，通过签名 URL 访问
- 多人协同需要配合 WebSocket 推送编辑状态

Q44: Activity/Flowable 工作流核心表

分类	表名	说明
流程定义	act_re_procdef	流程定义
运行实例	act_ru_execution	执行实例
运行实例	act_ru_task	当前任务/节点
历史	act_hi_procinst	历史流程实例
历史	act_hi_taskinst	历史任务实例
历史	act_hi_actinst	历史活动实例
身份	act_id_user	用户
身份	act_id_group	用户组

Q45: 为什么 Spring Boot 的 jar 可以是 Web 应用

普通 jar 没有 Servlet 容器。Spring Boot jar 里嵌入了 Tomcat（`spring-boot-starter-web` 中 `tomcat-embed-core`），`JarLauncher` 启动时创建内嵌 Tomcat 实例、注册 `DispatcherServlet` 并监听端口。

jar 包结构:

```
BOOT-INF/  
  lib/          → 依赖 jar (含 tomcat-embed)  
  classes/      → 应用代码  
META-INF/  
  MANIFEST.MF   → Main-Class: JarLauncher  
org.springframework.boot.loader/ → Spring Boot 类加载器
```

十、快速索引

关键词	对应问题
HashMap/扩容	Q1
String/常量池	Q2
int vs Integer	Q3
run vs start	Q4
单例/DCL	Q5
去重	Q6
list转map	Q7
线程安全	Q8
线程池	Q9
死锁	Q10
Bean 线程安全	Q12
@Transactional	Q13
Spring 事务	Q14
Spring MVC vs Boot	Q15
@SpringBootApplication	Q16
Spring Security	Q17
Feign	Q18
微服务体系	Q19
B+ 树	Q20
回表	Q21
最左匹配/索引失效	Q22
INT(10) vs INT(11)	Q23
ORDER BY 失效	Q24
大表查询优化	Q25
千万级数据	Q26
分库分表	Q27
Redisson/看门狗	Q28
报告生成场景	Q29
缓存不一致	Q30
消息堆积	Q31
重复消费/幂等	Q32
unack	Q33
MyBatis 一对多	Q34

关键词	对应问题
接口慢排查	Q35
QPS 200	Q36
POI OOM	Q37
积分清零	Q38
重试机制	Q39
接口安全	Q40
Arthas trace	Q41
Linux 排查	Q42
WPS 在线编辑	Q43
Activity 表	Q44
Spring Boot jar	Q45

十一：补充

Q46: 分布式调度

为什么需要分布式调度？

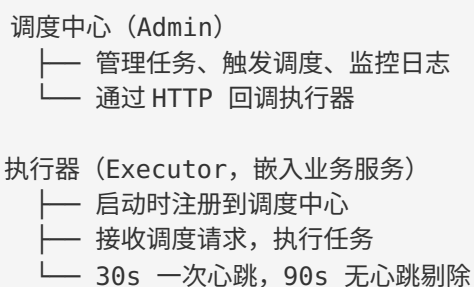
单机定时任务（如 `@Scheduled`）在多节点部署时会产生以下问题：

- **重复执行**：多台机器同时触发同一任务（如每天凌晨结算，每个节点都跑一遍）
- **无法动态扩缩**：不能灵活分配任务到不同节点
- **无失败重试/告警**：任务挂了没人知道，缺少运维能力
- **无分片能力**：大数据量任务无法拆分并行处理

主流方案对比：

特性	Quartz	XXL-Job	Elastic-Job
分布式协调	JDBC 行锁	中心化调度中心	ZooKeeper
分片	不支持	支持（广播+分片参数）	原生支持
管理界面	无（需自建）	自带 Web 控制台	自带控制台
动态添加任务	不支持	支持（控制台操作）	支持
失败重试	需自实现	内置	内置
社区活跃度	一般	极高	停更（已捐赠 Apache）

XXL-Job 核心原理：



分片广播机制：

当任务设置为"广播+分片"模式时，调度中心将所有执行器实例的路由策略置为"分片广播"，每个执行器收到参数 `shardIndex`（当前分片序号）和 `shardTotal`（总分片数）。

```

// 3 台机器, 处理 300 万条数据
// 执行器 0: id % 3 = 0 的数据 → 100万条
// 执行器 1: id % 3 = 1 的数据 → 100万条
// 执行器 2: id % 3 = 2 的数据 → 100万条
@XxlJob("cleanExpiredData")
public void cleanExpiredData() {
    int shardIndex = XxlJobHelper.getShardIndex(); // 0/1/2
    int shardTotal = XxlJobHelper.getShardTotal(); // 3
    List<Long> ids = mapper.getIdsByShard(shardIndex, shardTotal);
    // 逐批处理...
}

```

实际场景（积分清零）： 100w 用户积分每年 1.1 清零 → XXL-Job 凌晨 2:00 触发，3 节点分片，每节点处理 ~33w 用户，单节点内 5000 条一批，分页更新。

Q47: 分布式事务

产生背景：

单体架构 → 微服务拆分后，一个"下单"操作可能跨 3 个服务（订单服务、库存服务、优惠券服务），每个服务有独立的数据库。单机 @Transactional 只能管自己的库，需要分布式事务保证跨服务的数据一致性。

理论基础：

CAP 定理：分布式系统无法同时满足 C（一致性）、A（可用性）、P（分区容错性），P 必须保证，只能在 C 和 A 之间权衡。

BASE 理论：

基本可用（Basically Available）：允许部分功能降级

软状态（Soft State）：允许系统存在中间状态

最终一致性（Eventually Consistent）：数据最终一致即可

方案一：2PC（两阶段提交）

阶段1（Prepare）：协调者问所有参与者——能提交吗？

- ├ 参与者执行 SQL，写 undo/redo 日志，但不 commit
- └ 回复 YES（可以提交）或 NO（失败）

阶段2（Commit/Rollback）：

- ├ 全部 YES → 协调者发 Commit → 所有参与者提交
- └ 任一 NO → 协调者发 Rollback → 所有参与者回滚

2PC 致命缺陷：

- **同步阻塞：** prepare 阶段锁资源一直不释放，时间窗口长
- **单点故障：** 协调者挂了，所有参与者卡住（参与者有超时机制但协调者没有）
- **数据不一致：** commit 阶段部分节点收到、部分没收到（网络分区）
- **实际很少使用，**只适用于数据库层面（XA 协议）

方案二：3PC（三阶段提交）

2PC 的改良版，在 Prepare 和 Commit 之间插入 PreCommit 阶段，同时给参与者也加了超时机制：

阶段1 (CanCommit) : 能提交吗? (只询问, 不做任何操作)

阶段2 (PreCommit) : 预执行 (写日志但不提交)

阶段3 (DoCommit) : 真正提交/回滚

改进:

- 参与者在 PreCommit 后超时未收到 DoCommit → **自动提交** (避免了 2PC 的无限等待)
- 代价: 可能产生数据不一致 (参与者超时后自动提交了, 但协调者决定回滚)

实际也很少使用, 2PC/3PC 都是刚性事务, 性能差。

方案三: TCC (Try-Confirm-Cancel)

核心思想: 把业务逻辑拆成三个方法, 由业务方自己实现。

Try (预留资源) → 冻结库存 100 件 (还没扣)

Confirm (确认) → 真正扣减库存 100 件

Cancel (回滚) → 解冻库存 100 件

流程:

1. TM 调所有参与者的 Try 方法
2. 全部 Try 成功 → TM 调所有 Confirm
3. 任一 Try 失败 → TM 调所有 Cancel

TCC 三大难题及解法:

问题	场景	解法
空回滚	Try 超时没到 → TM 触发 Cancel → 后来 Try 才到达	Cancel 执行前检查是否 Try 过, 没 Try 过直接返回成功
悬挂	Cancel 先到 → Try 后到	Cancel 记录回滚标记, Try 先查标记, 已回滚则不执行
幂等	Confirm/Cancel 被重复调	用 xid + branchId 做唯一键, 防重复执行

示例 (Seata TCC) :

```
@TwoPhaseBusinessAction(name = "deductStock", commitMethod = "confirm", rollbackMethod = "cancel")
public boolean tryDeduct(@BusinessActionContextParameter(paramName = "skuId") Long skuId,
                        @BusinessActionContextParameter(paramName = "count") Integer count) {
    // Try: 冻结库存
    return stockService.freeze(skuId, count);
}

public boolean confirm(BusinessActionContext ctx) {
    // Confirm: 真正扣减
    Long skuId = (Long) ctx.getActionContext("skuId");
    Integer count = (Integer) ctx.getActionContext("count");
    return stockService.deduct(skuId, count);
}

public boolean cancel(BusinessActionContext ctx) {
    // Cancel: 解冻
    Long skuId = (Long) ctx.getActionContext("skuId");
```

```
Integer count = (Integer) ctx.getActionContext("count");
return stockService.unfreeze(skuId, count);
}
```

优缺点：

- **优点：**性能高（Try 阶段只锁业务资源，不锁数据库行），对数据库无侵入
- **缺点：**业务侵入性强，每个服务需要写三个方法，设计复杂

方案四：事务消息（最终一致性）

核心思路：不追求强一致，通过消息队列保证"最终"一致。

RocketMQ 事务消息流程：

1. 生产者发 Half 消息（半消息，对消费者不可见）
2. 生产者执行本地事务
 - └─ 成功 → Commit → 消息对消费者可见 → 消费者消费
 - └─ 失败 → Rollback → 消息删除
3. 生产者宕机/超时 → MQ 回调 CheckListener 查询本地事务状态
 - └─ 已提交 → Commit
 - └─ 已回滚 → Rollback

场景示例（下单扣库存）：

1. 订单服务发 Half 消息 "订单创建成功"
2. 订单服务执行本地事务：插入订单表，状态=待支付
3. 成功 → Commit 消息
4. 库存服务消费消息 → 扣库存
5. 如果库存不足 → 发退款补偿消息 → 订单服务标记订单已取消

优缺点：

- **优点：**最终一致性，系统解耦，吞吐量高
- **缺点：**中间状态不可见（用户看到"处理中"），需要补偿机制

四种方案对比总结：

方案	一致性	性能	侵入性	适用场景
2PC	强一致	极差	低	几乎不用
3PC	强一致	差	低	几乎不用
TCC	最终一致	高	高	核心系统、对性能要求高的场景
事务消息	最终一致	高	中	异步解耦场景（下单→扣库存→发券）
Seata AT	最终一致	中	极低	不想改业务代码，需要快速接入

Q48: Prototype Bean 注入到 Singleton 中为什么会退化？

问题本质：

```
@Component
@Scope("prototype")
public class PrototypeBean { }

@Component
```

```
public class SingletonBean {
    @Autowired
    private PrototypeBean prototypeBean; // 只注入一次！
}
```

Spring 的 Singleton Bean 在容器启动时就创建好了，它的依赖也会在那一刻注入。换句话说，SingletonBean 里的 prototypeBean 在整个 Spring 生命周期里**永远是同一个对象**——第一次注入后不会再换了。

为什么用 @Lookup 能解决？

```
@Component
public class SingletonBean {
    @Lookup
    public PrototypeBean getPrototypeBean() {
        return null; // Spring 用 CGLIB 重写这个方法
    }
}
```

Spring 通过 CGLIB 动态代理生成 SingletonBean 的子类，重写 @Lookup 注解的方法，每次调用都走 BeanFactory.getBean() 重新获取新的 Prototype 实例。

其他解法：

方式	说明
@Lookup	Spring 原生支持，推荐
ApplicationContext.getBean()	直接拿容器，但耦合 Spring 框架
ObjectFactory / Provider	@Autowired private ObjectFactory<PrototypeBean> factory; 然后 factory.getObject()
把"会变的状态"抽出去	换个思路——不要依赖每次 new 对象，用 ThreadLocal 或 Redis 存状态，Bean 保持无状态

核心心法：在 Spring 容器里，不要期望注入的 Prototype Bean 会"自动刷新"，Spring 只管注入那一次。需要"每次用新的"，就得自己调用获取。

Q49: CompletableFuture 异常处理

内容如上文代码示例，这里补充核心结论

allOf().join() 的坑：

- allOf 返回的 CompletableFuture 只要所有任务**结束**（不管成功还是异常）就算完成
- join() 不会因为某个子任务异常而抛异常
- 子任务的异常会被**静默吞掉**

三种处理方式对比：

方式	适用场景	缺点
每个 future 内部 try-catch + 收集异常 handle() 统一收尾	需要精确知道哪个任务失败	代码冗余
exceptionally() 只处理异常	每个任务都有明确的成功/失败结果	返回值类型统一
orTimeout() + completeOnTimeout()	只需要降级兜底	正常逻辑和异常逻辑分开
	超时控制	Java 9+

```
// Java 9 超时控制
CompletableFuture<String> future = CompletableFuture
    .supplyAsync(() -> callRemoteService())
    .orTimeout(3, TimeUnit.SECONDS)
    .completeOnTimeout("降级默认值", 3, TimeUnit.SECONDS);
```